Reporte

Nombre: Roberto Carlos Sánchez Flores Donovan Adrian Serrato Lores Laura Sofía Vargas Ramírez	**Matrícula**: 2151262 2154304 2152053
Nombre del curso: Componentes de lógica de negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 3	**Actividad**: Diseño arquitectural del sistema
Fecha: 15/10/2026	

Bibliografia (Formato APA):

Documentos dados por el docente (guía «Diseño arquitectural del sistema con IA»).

Equipo 12. (2026). BRD, User Stories, BPMN y Glosario de CleanPro [Documentos de la Unidad 2].

Evans, E. (2003). Domain-Driven Design: Tackling complexity in the heart of software. Addison-Wesley.

Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.

Martin, R. C. (2017). Clean architecture: A craftsman's guide to software structure and design. Prentice Hall.

Objetivo:

Convertir el análisis de negocio de CleanPro elaborado en la Unidad 2 (BRD, User Stories, BPMN y Glosario) en una propuesta de arquitectura de software que separe el dominio, la aplicación y la infraestructura. Para ello se seleccionan los casos de uso del proceso principal, se compara y justifica el patrón de lógica de negocio, se modela el dominio con lenguaje ubicuo, Aggregates y Domain Events, se diseñan la Service Layer, los Repository, la Unit of Work y una separación básica de Commands y Queries, y se revisa todo con los principios SOLID. Las decisiones se documentan con diagramas UML, registros de decisión de arquitectura (ADR), una matriz de trazabilidad y una bitácora del uso de inteligencia artificial.

Procedimiento:

CleanPro — Diseño arquitectural del sistema

CleanPro presta servicios de limpieza residencial y empresarial. El precio depende de las características del inmueble, la frecuencia y los extras; el proceso abarca desde la solicitud de cotización hasta la ejecución del servicio, el registro de incidencias y la facturación. El BRD identificó cuatro problemas: los cambios no llegan a administración (PA-01), hay errores de facturación (PA-02), las cuadrillas se asignan a mano (PA-03) y hay poca evidencia de incidencias y extras (PA-04). La arquitectura se diseñó para atender esos problemas sin inventar reglas nuevas.

Continuidad con la Unidad 2

Se trabajó con la misma empresa y el mismo alcance. Los productos de la Unidad 2 se usaron así:

Insumo de la Unidad 2	Cómo se utilizó en la Unidad 3
BRD	Alcance (ALC-01 a ALC-10), necesidades BN-01 a BN-10, 15 reglas RB, 6 restricciones RES, 6 supuestos SUP y situaciones excepcionales. Es la evidencia de cada decisión.
User Stories y criterios Gherkin	Selección de los casos de uso y de los escenarios de prueba (US-01 a US-22).
BPMN (v2)	Secuencia del proceso, gateways de decisión y los tres fines posibles (rechazada, no realizado, completado y facturado).
Glosario (30 términos)	Lenguaje ubicuo: los nombres de clases, métodos y diagramas salen del glosario.
Matriz BN → US	Se amplió con reglas, casos de uso, componentes, ADR y pruebas.

Etapa 1. Preparar el caso

Proceso principal elegido: ciclo de vida del servicio de limpieza — solicitud de cotización → estimación y envío → respuesta del cliente → programación y asignación de cuadrilla → ejecución con incidencias → facturación. Se seleccionaron cinco casos de uso que incluyen flujos normales y flujos con rechazo:

ID	Caso de uso	Actor	User Stories	Reglas	Resultado esperado
CU-01	Generar y enviar cotización	Coordinador (a partir de la solicitud del Cliente)	US-01, US-06, US-07	RB-01, RB-03, RB-15, SUP-06	Cotización con precio calculado, en estado Pendiente y con vigencia de 5 días hábiles.
CU-02	Responder cotización	Cliente	US-02, US-22	RB-02, RB-08, RES-05	Cotización Aceptada (habilita la programación), Rechazada o Vencida.

ID	Caso de uso	Actor	User Stories	Reglas	Resultado esperado
CU-03	Programar servicio y asignar cuadrilla	Coordinador	US-08, US-09, US-03	RB-02, RB-04, RES-01, RES-02, SUP-01, SUP-02	Servicio Programado con fecha, hora, cuadrilla y supervisor; cliente notificado.
CU-04	Registrar incidencia durante el servicio	Supervisor	US-15, US-16, US-11, US-19	RB-06, RB-11, RB-12, RB-14, RES-06, SUP-03	Incidencia asociada al servicio y notificada a administración; puede generar un cargo o dejar el servicio No realizado.
CU-05	Facturar servicio	Administración	US-20, US-05	RB-07, RB-11, RB-14, RES-04, SUP-04	Factura única vinculada al servicio y a sus incidencias, con estado de pago Pendiente.

Tabla 1. *Casos de uso seleccionados.*

Flujos de rechazo y alternos considerados:

- **CU-01:** un extra que no existe en el catálogo de extras no se puede cotizar (SUP-06).
- **CU-02:** si el cliente responde después de 5 días hábiles la cotización queda Vencida y el coordinador debe generar una nueva (RB-08, US-22); el cliente también puede rechazarla.
- **CU-03:** no se programa si la cotización no fue aceptada (RB-02, RES-02) ni si la cuadrilla está ocupada a esa fecha y hora (RES-01).
- **CU-04:** un servicio adicional en sitio o un material especial generan un cargo que requiere autorización del coordinador (RB-12, RB-14); si el cliente no permite el acceso, tras 15 minutos el servicio queda No realizado (RB-11, RES-06).
- **CU-05:** no se factura un servicio que no esté Terminado o No realizado (RB-07), ni con cargos sin autorizar (RB-14), ni dos veces (SUP-04).

Los cinco casos de uso aparecen en el BPMN (Solicitar cotización, Estimar servicio, Enviar cotización, ¿Cliente acepta?, Programar servicio, Asignar cuadrilla, ¿Cliente permite el acceso?, Registro de incidencia y Registrar servicio / facturación) y en la matriz BN → US.

Hallazgos de continuidad

Al cruzar el BRD, las User Stories, el glosario y el BPMN se encontraron los siguientes hallazgos. No son reglas nuevas: aclaran o corrigen el análisis de la Unidad 2.

ID	Hallazgo	Evidencia	Ajuste
H-01	Contradicción: RB-07 dice que solo se factura un servicio terminado, pero RB-11 factura el 50% de un servicio no realizado.	RB-07 vs RB-11, US-16	Se factura un servicio Terminado o No realizado.

ID	Hallazgo	Evidencia	Ajuste
H-02	Los estados del servicio del BRD no incluyen «no realizado».	RB-11, BPMN	Se agrega el estado NoRealizado.
H-03	Los estados de la cotización no incluyen «vencida» ni el momento antes del envío.	US-22, RB-08, glosario #6	Solicitada → Pendiente → Aceptada / Rechazada / Vencida.
H-04	US-06 pide calcular duración y personal, pero el BRD solo da la fórmula del precio.	US-06 vs RB-03	Supuesto aceptado por el equipo: el coordinador captura duración y personal; el sistema calcula el precio.
H-05	No queda claro si el 10% de descuento recurrente incluye los extras.	RB-03, glosario #5	Supuesto aceptado por el equipo: el descuento se aplica solo a superficie × tarifa.
H-06	Con frecuencia recurrente no se define si una cotización genera varios servicios.	RB-03, SUP-01	Supuesto: cada programación crea un solo servicio (trabajo posterior).

Historias fuera del diseño detallado: US-04 Reprogramación (RB-09) y US-10 Reasignar personal (RB-10) quedan como trabajo posterior para no exceder cinco casos de uso; US-12 Recalcular cotización (RB-13) se registra como incidencia; US-13 y US-14 (inicio y terminación) se modelan como métodos del Servicio porque CU-04 y CU-05 dependen de esos estados; US-17 y US-21 se usan como consultas en la etapa de CQRS.

Etapa 2. Elegir el patrón de lógica de negocio

Se compararon Transaction Script (un procedimiento por operación), Table Module (una clase por tabla que opera sobre conjuntos de registros) y Domain Model (objetos del negocio que guardan sus datos y sus reglas).

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Complejidad de reglas	El cálculo de precio se repetiría al cotizar y al recalcular; riesgo de copiar y pegar.	Las excepciones de ejecución dependen del estado de un servicio concreto, no de la tabla.	El cálculo vive en una sola política y cada excepción es un comportamiento del Servicio.	RB-03, RB-11 a RB-14; 15 reglas y 6 excepciones
Cambios esperados	Cambiar tarifa, descuento, umbral o vigencia obliga a buscar el valor en varios scripts.	Igual: los valores quedan regados en los métodos de cada tabla.	Tarifas, descuento y vigencia son políticas o Value Objects; se cambian en un solo lugar.	RB-03, RB-08, RB-13

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Interacción entre conceptos	Cada script arma el total consultando cotización, servicio, incidencias y cargos (causa de PA-02).	Coordinar varias tablas desde cada operación; la relación queda implícita.	Relación explícita: el Servicio conoce su Cotización e Incidencias y la Factura se arma desde el Servicio.	RES-04, BN-08, glosario #30, PA-02
Estados e invariantes	Los estados se validan con if en cada script; es fácil olvidar uno.	Un estado por registro no encaja en un módulo que trabaja con todo el conjunto.	Cada Aggregate protege sus transiciones (no programar sin aceptación, no facturar sin terminación).	RB-02, RB-04, RB-07, RB-14, RES-02, RES-03
Trazabilidad y auditoría	Hay que reconstruir la evidencia consultando tablas.	Similar a Transaction Script.	Cada cambio importante produce un Domain Event (por ejemplo IncidenciaRegistrada) .	PA-01, PA-04, BN-06, CE-02, CE-04

Tabla 2. *Comparación de patrones con criterios del caso.*

Patrón	Ventaja en CleanPro	Riesgo o costo en CleanPro
Transaction Script	Rápido de programar; bastaría para un CRUD de cotizaciones.	Con 15 reglas y 6 excepciones las validaciones se duplican y es fácil facturar mal (repetiría PA-02).
Table Module	Natural para reportes por estado (REP-01 a REP-05).	Las reglas por servicio individual (acceso denegado, cargos por autorizar) no encajan en una clase por tabla.
Domain Model	Reglas e invariantes en un solo lugar, estados protegidos y el glosario en el código.	Más clases, curva de aprendizaje y necesidad de Repository y Unit of Work.

Decisión: Domain Model para las operaciones que cambian el estado (CU-01 a CU-05), combinado de forma delimitada con consultas simples para los reportes (REP-01 a REP-05) y con el catálogo de extras como datos de referencia. **Problema que resuelve:** CleanPro factura con errores porque el monto depende de la cotización, del estado del servicio y de los cargos de cada incidencia; en el Domain Model la factura se arma desde un Servicio que ya validó sus incidencias y cargos. **Costo aceptado:** más clases y diseño previo, y el uso obligatorio de Repository y Unit of Work; se acepta porque un error de cobro cuesta más y la facturación es prioridad alta en el BRD.

Evidencias de la Unidad 2: (1) RB-03 + RB-13: el cálculo con variantes y el recálculo necesitan una sola política de precio; (2) RB-07, RB-11, RB-14 y RES-03: invariantes de

estado antes de facturar; (3) PA-01, PA-02, PA-04 y CE-01, CE-02: los problemas vienen de información dispersa; (4) el BPMN tiene cuatro decisiones y tres fines distintos. La decisión quedó registrada en el ADR-001.

Etapa 3. Construir el mapa de dominio

El lenguaje y las reglas de la Unidad 2 se transformaron en un mapa de responsabilidades con tres Aggregates (Cotización, Servicio y Factura), sus Value Objects, tres Domain Services y diez Domain Events. No se convirtió cada dato en una entidad: Cliente, Coordinador y Supervisor solo se referencian por Id y el catálogo de extras es un dato de referencia. El diagrama completo se muestra en la Figura 1.

Lenguaje ubicuo

Término del negocio (glosario U2)	Elemento del dominio	Nombre en código
Cotización (#1)	Aggregate Root	Cotizacion
Solicitud de cotización (#2)	Estado inicial de la Cotización	EstadoCotizacion.Solicitada
Tarifa base (#3) / Frecuencia (#5)	Regla del Domain Service / Value Object	CalculadoraDePrecio, Frecuencia
Extra (#4) / Catálogo de extras (#24)	Value Object / dato de referencia	ExtraSolicitado, ExtraDelCatalogo
Vigencia de la cotización (#6)	Value Object	Vigencia
Programación del servicio (#13)	Value Object	Programacion
Cuadrilla (#9) / Asignación (#14)	Referencia por Id + Domain Service	CuadrillaId, DisponibilidadDeCuadrilla
Ejecución del servicio (#15)	Aggregate Root y estados	Servicio, EstadoServicio
Registro de inicio / terminación (#16, #17)	Comportamientos del Servicio	RegistrarInicio(), RegistrarTerminacion()
Incidencia (#18)	Entity dentro del Aggregate Servicio	Incidencia
Cliente no permite acceso (#21) / Cargo de cancelación (#22)	Comportamiento + concepto de factura	RegistrarAccesoDenegado(), ConceptoFacturado
Servicio adicional en sitio (#23) / Material especial (#25)	Tipo de incidencia con cargo	TipoDeIncidencia, CargoAdicional
Factura (#28)	Aggregate Root	Factura
Trazabilidad (#30)	Referencias por Id + Domain Events	CotizacionId, ServicioId, eventos

Qué representa cada elemento en el negocio

- **Entidades:** Cotización, Servicio, Incidencia y Factura tienen identidad y ciclo de vida: hay que distinguirlas aunque cambien sus datos (una cotización vence, un servicio pasa por varios estados, el coordinador autoriza una incidencia concreta, una factura se paga).

- **Value Objects:** DatosDelInmueble, Frecuencia, ExtraSolicitado, EstimacionOperativa, Vigencia, Programacion, CargoAdicional, ConceptoFacturado y Dinero se definen por sus valores; si cambian se reemplazan. Dinero evita sumar números sueltos y redondear distinto (PA-02).
- **Domain Services:** CalculadoraDePrecio aplica RB-03 y se reutiliza en el recálculo (RB-13); DisponibilidadDeCuadrilla necesita ver otros servicios (RES-01); GeneradorDeFactura involucra a Servicio y Factura (RB-07, RB-11, RB-14, SUP-04).
- **Domain Events:** hechos que ya ocurrieron y le interesan a otro rol. El más importante es IncidenciaRegistrada → Administración, que resuelve PA-01.

Fichas de los Aggregates

Campo	Cotización	Servicio	Factura
Raíz	Cotizacion	Servicio	Factura
Contiene	DatosDelInmueble, Frecuencia, ExtraSolicitado (0..*), EstimacionOperativa, Vigencia, Dinero	Programacion, Incidencia (0..*), CargoAdicional (0..1 por incidencia); referencias a Cotización, Cuadrilla y Supervisor	ConceptoFacturado (1..*), EstadoDePago; referencia al Servicio
Responsabilidades	Guardar el precio calculado, controlar el envío, la respuesta del cliente y el vencimiento.	Controlar el ciclo de ejecución, registrar incidencias y cargos, aplicar el acceso denegado e indicar si ya se puede facturar.	Guardar los conceptos cobrados, su total y el estado de pago.
Invariantes	Solo se envía con precio (RB-01, RB-03); solo se responde si está Pendiente (US-02); fuera de 5 días hábiles queda Vencida (RB-08); solo el coordinador la estima (RB-15).	Se crea solo desde una cotización Aceptada y con cuadrilla (RB-02, RB-04); inicio solo si está Programado y terminación solo si está En ejecución (RB-05, RES-03); acceso denegado tras 15 min (RB-11, RES-06); facturable solo si Terminado o NoRealizado y sin cargos pendientes (RB-07, RB-14, H-01).	Total = suma de conceptos (RES-04); una factura por servicio (SUP-04); se emite con pago Pendiente.
Eventos	CotizacionEnviada, CotizacionAceptada, CotizacionRechazada, CotizacionVencida	ServicioProgramado, IncidenciaRegistrada, CargoAdicionalAutorizado, ServicioTerminado, ServicioNoRealizado	FacturaEmitida

Tabla 3. *Fichas de los Aggregates.*

Domain Events

Evento	Lo publica	Le interesa a	Regla / historia
CotizacionEnviada	Cotizacion	Cliente	US-07

Evento	Lo publica	Le interesa a	Regla / historia
CotizacionAceptada / Rechazada	Cotizacion	Coordinador	RB-02, US-02
CotizacionVencida	Cotizacion	Coordinador (nueva cotización)	RB-08, US-22
ServicioProgramado	Servicio	Cliente	US-03
IncidenciaRegistrada	Servicio	Administración	RB-06, US-19, BN-06, PA-01
CargoAdicionalAutorizado	Servicio	Administración / facturación	RB-12, RB-14
ServicioTerminado / ServicioNoRealizado	Servicio	Administración (puede facturar)	US-14, RB-07, RB-11
FacturaEmitida	Factura	Cliente	US-05

Glosario actualizado

Término	Cambio	Definición	Origen
Estado de la cotización	Corregido	Solicitada, Pendiente, Aceptada, Rechazada o Vencida.	H-03, US-22
Estado del servicio	Corregido	Programado, En ejecución, Terminado o No realizado.	H-02, RB-11
Servicio no realizado	Nuevo	Servicio que no se ejecutó porque el cliente no permitió el acceso; se factura solo el cargo de cancelación.	RB-11, H-01
Cargo adicional	Nuevo	Monto de un servicio adicional en sitio o material especial que el coordinador debe autorizar antes de facturar.	RB-12, RB-14
Estimación operativa	Nuevo	Duración y personal que captura el coordinador al estimar.	US-06, H-04
Precio acordado	Nuevo	Precio de la cotización aceptada que se copia al servicio al programarlo.	RB-02
Concepto facturado	Nuevo	Cada renglón de la factura: servicio, cargo de incidencia o cargo de cancelación.	RES-04

Decisiones dudosas o descartadas

Decisión	Alternativa descartada	Por qué
Incidencia dentro del Aggregate Servicio (ADR-003)	Incidencia como Aggregate propio	RES-03 y RB-14 necesitan ver juntos al servicio y sus incidencias en la misma transacción. Costo: el servicio se carga completo.
Factura como Aggregate separado	Factura dentro de Servicio	Tiene su propio ciclo (pago) y la gestiona otro rol (RB-15).

Decisión	Alternativa descartada	Por qué
Cuadrilla solo como referencia	Aggregate Cuadrilla	La reasignación de personal (US-10) quedó fuera de los cinco casos de uso.
Catálogo de extras como dato de referencia	Aggregate Catálogo	No tiene ciclo de vida ni reglas en estos casos de uso (SUP-06).
«Una factura por servicio» en GeneradorDeFactura	Validarla en Factura o en el Application Service	Una factura no puede ver otras facturas; primero se pensó en el Application Service, pero la revisión SOLID (S-6) la movió al dominio.

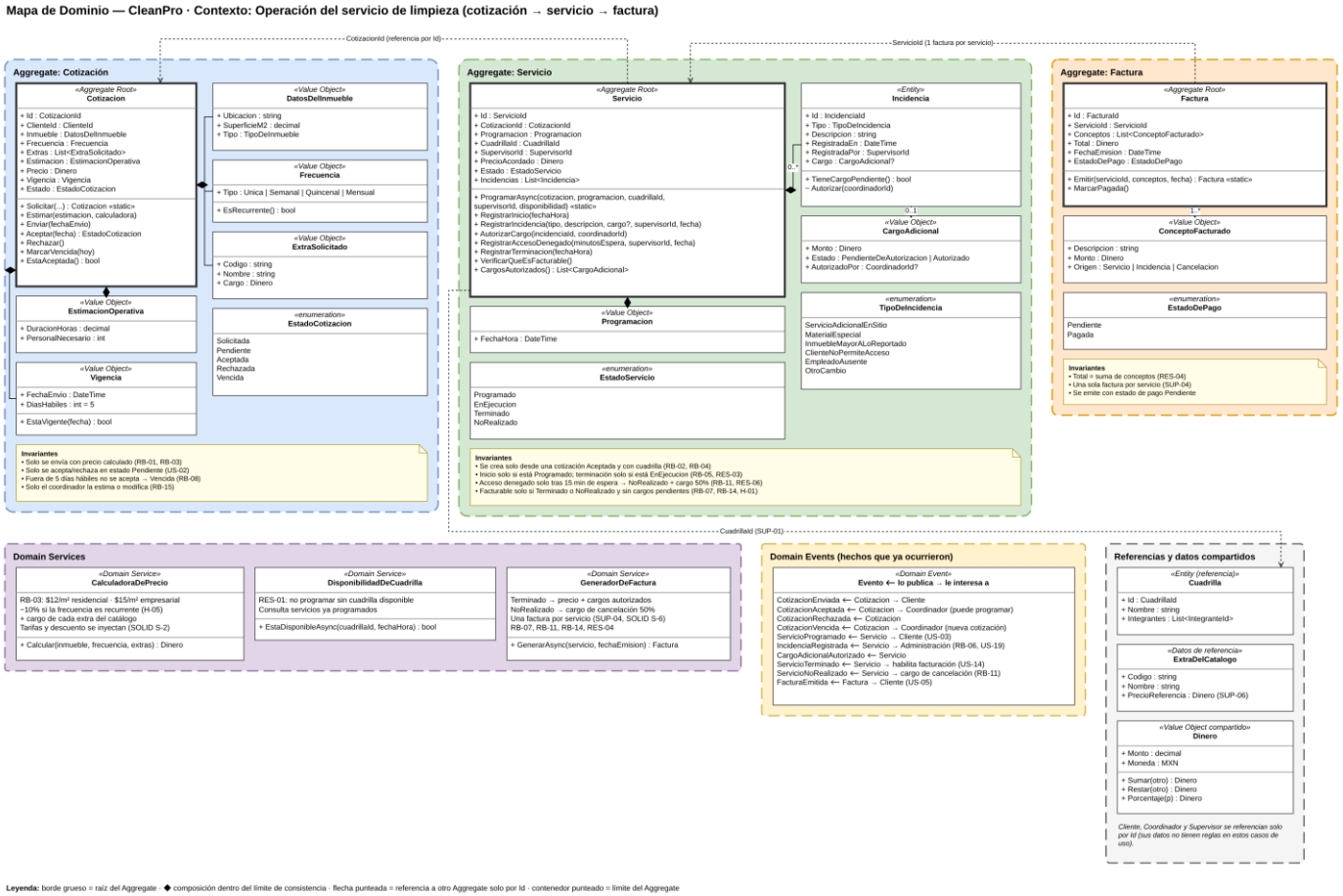


Figura 1. *Mapa de dominio de CleanPro (archivo fuente diagrams/src/01-mapa-dominio.drawio).*

Etapa 4. Diseñar la Service Layer

Los Application Services se agruparon por actor (coordinador, supervisor, administración) porque RB-15 separa quién puede hacer qué. Cada método es una intención completa del usuario y solo coordina: recibe el Command, obtiene los Aggregates, invoca el dominio, confirma la Unit of Work y devuelve un resultado.

Servicio y método (caso de uso)	Entrada → salida	Colaboradores	Reglas que delega
CotizacionAppService .SolicitarCotizacion (CU-01)	SolicitarCotizacionCommand → CotizacionId	ICatalogoDeExtras, ICotizacionRepository, IUnitOfWork	SUP-06 → catálogo; datos del inmueble → DatosDelInmueble
CotizacionAppService .EstimarYEnviarCotizacion (CU-01)	EstimarYEnviarCotizacionCommand → CotizacionEnviadaDto	ICotizacionRepository, CalculadoraDePrecio, IUnitOfWork	RB-03 → CalculadoraDePrecio; RB-01, RB-08 → Cotizacion.Enviar
CotizacionAppService .ResponderCotizacion (CU-02)	ResponderCotizacionCommand → EstadoCotizacion	ICotizacionRepository, IUnitOfWork	RB-08, RES-05 → Cotizacion.Aceptar / Rechazar
ProgramacionAppService .ProgramarServicio (CU-03)	ProgramarServicioCommand → ServicioId	ICotizacionRepository, IServicioRepository, DisponibilidadDeCuadrilla, IUnitOfWork	RB-02, RB-04 → Servicio.ProgramarAsync; RES-01 → DisponibilidadDeCuadrilla
EjecucionServicioAppService .RegistrarIncidencia (CU-04)	RegistrarIncidenciaCommand → IncidenciaId	IServicioRepository, ICatalogoDeExtras, IUnitOfWork	RB-06, RB-12, RB-14, SUP-03 → Servicio.RegistrarIncidencia
EjecucionServicioAppService .RegistrarAccesoDenegado (CU-04)	RegistrarAccesoDenegadoCommand → EstadoServicio	IServicioRepository, IUnitOfWork	RB-11, RES-06 → Servicio.RegistrarAccesoDenegado
EjecucionServicioAppService .AutorizarCargoAdicional (CU-04)	AutorizarCargoAdicionalCommand → (sin salida)	IServicioRepository, IUnitOfWork	RB-12, RB-14, RB-15 → Servicio.AutorizarCargo
FacturacionAppService .FacturarServicio (CU-05)	FacturarServicioCommand → FacturaId	IServicioRepository, GeneradorDeFactura, IFacturaRepository, IUnitOfWork	RB-07, RB-11, RB-14, SUP-04 → GeneradorDeFactura; RES-04 → Factura

Tabla 4. *Application Services: un método por intención del usuario.*

Métodos de soporte: EjecucionServicioAppService.RegistrarInicio y RegistrarTerminacion (US-13, US-14) solo llaman al dominio; existen porque CU-04 y CU-05 dependen de esos estados. Las notificaciones no las envía ningún servicio: salen de los Domain Events al confirmar la Unit of Work.

Pregunta de revisión	Respuesta del diseño
¿El nombre del método expresa una acción del negocio?	Sí: Solicitar, Estimar y enviar, Responder, Programar, Registrar incidencia, Autorizar cargo, Facturar. No hay métodos genéricos como Guardar.
¿Coordina o contiene decisiones del dominio?	Coordina. FacturarServicio no decide el monto ni si se puede facturar; lo hacen GeneradorDeFactura y Servicio.
¿Depende de interfaces o de la base de datos?	Solo de interfaces: repositorios, IUnitOfWork, IPublicadorDeEventos e ICatalogoDeExtras.
¿Tiene un inicio y un final transaccional claro?	Empieza al recibir el Command y termina en IUnitOfWork.ConfirmarAsync(); si algo falla antes, no se guarda nada.
¿Los errores se explican en términos del negocio?	Sí: CotizacionNoAceptada (RB-02), CuadrillaNoDisponible (RES-01), TiempoDeEsperaInsuficiente (RES-06), CargosPendientesDeAutorizacion (RB-14), ServicioNoFacturable (RB-07), FacturaYaEmitida (SUP-04), ExtraNoExisteEnCatalogo (SUP-06).

Etapa 5. Separar persistencia y transacciones

Los Repository son interfaces del dominio, hablan en términos del negocio y solo devuelven o guardan Aggregates completos. Los reportes no van aquí, sino en las consultas del lado Query.

Repository	Necesidad del dominio	Métodos propuestos	Objeto que recupera o guarda
ICotizacionRepository	Recuperar una cotización para estimarla, responderla o programarla	ObtenerPorIdAsync, AgregarAsync	Cotizacion
IServicioRepository	Recuperar un servicio con sus incidencias; saber si una cuadrilla ya está ocupada	ObtenerPorIdAsync, AgregarAsync, CuadrillaTieneServicioEnAsync	Servicio (con Incidencia y CargoAdicional)
IFacturaRepository	Guardar la factura y evitar facturar dos veces	AgregarAsync, ExisteFacturaParaServicioAsync	Factura
ICatalogoDeExtras	Consultar el precio de referencia de un extra (solo lectura)	ObtenerPorCodigoAsync	ExtraDelCatalogo

Implementación en infraestructura: la propuesta real es EF Core con un CleanProDbContext, donde la Unit of Work es SaveChangesAsync dentro de una transacción y los eventos se guardan en una tabla Outbox. Como el caso es ficticio, el código base usa implementaciones en memoria (...RepositoryEnMemoria,

UnitOfWorkEnMemoria) que respetan el mismo contrato: nada nuevo se guarda ni se publica hasta confirmar.

Unit of Work: cada transacción modifica un solo Aggregate; puede leer otros, pero no cambiarlos. Los Domain Events se guardan en la misma transacción y se publican después de confirmar.

Caso de uso	Cambios incluidos en la misma transacción	Si ocurre un error
CU-01	Cotización (precio, estimación, vigencia, estado Pendiente) + evento CotizacionEnviada	No cambia el estado; el cliente no recibe una cotización a medias.
CU-02	Estado de la cotización + evento	La cotización sigue Pendiente.
CU-03	Nuevo Servicio (la Cotización solo se lee) + evento ServicioProgramado	No se crea el servicio; la cuadrilla queda libre.
CU-04	Servicio + nueva Incidencia + CargoAdicional + evento IncidenciaRegistrada	No queda una incidencia sin aviso ni un aviso sin incidencia (PA-01).
CU-05	Nueva Factura (el Servicio solo se lee) + evento FacturaEmitida	No se emite la factura.

Riesgo identificado: dos coordinadores podrían asignar la misma cuadrilla a la misma hora al mismo tiempo. En la implementación real se resolvería con una restricción única o con concurrencia optimista (ADR-002).

Etapa 6. Separar Commands y Queries

Los Commands cambian el estado y pasan por el dominio; el Handler de cada Command es el método del Application Service que lo recibe, para no duplicar clases. Las Queries solo leen y devuelven DTOs planos que nunca se usan para escribir. No se crearon bases de datos separadas.

Tipo	Nombre y datos de entrada	Handler	Resultado	Justificación
Command	ResponderCotizacionCommand (cotizacionId, acepta, fechaRespuesta)	CotizacionAppService .ResponderCotizacion	Nuevo estado	Cambia el estado y protege RB-02 / RB-08
Command	ProgramarServicioCommand (cotizacionId, fechaHora, cuadrillaId, supervisorId)	ProgramacionAppService .ProgramarServicio	ServicioId	Crea un Servicio; protege RB-02, RB-04, RES-01
Command	RegistrarIncidenciaCommand (servicioId, supervisorId, tipo, descripción, extra o monto)	EjecucionServicioAppService .RegistrarIncidencia	IncidenciaId	Cambia el Servicio y dispara la notificación (RB-06)
Command	FacturarServicioCommand (servicioId, fechaEmision)	FacturacionAppService .FacturarServicio	FacturaId	Crea la Factura; protege RB-07, RB-14, SUP-04
Query	ServiciosProgramadosQuery (desde, hasta)	ServiciosProgramados QueryHandler	List<ServicioProgramadoDto>	REP-02 y US-03: agenda sin cargar Aggregates

Tipo	Nombre y datos de entrada	Handler	Resultado	Justificación
Query	ServiciosDeCuadrillaQuery (cuadrillaId, fecha)	ServiciosDeCuadrilla QueryHandler	List<Servicio AsignadoDto >	US-17 y REP-05: la cuadrilla solo necesita dónde y cuándo
Query	FacturacionPorServicioQuery (estadoDePago opcional)	FacturacionPorServicio QueryHandler	List<Facturaci onDto>	REP-04 y US-21: cruza servicio, incidencias y factura

Dónde no hace falta separar: el catálogo de extras es un listado simple de referencia; un CRUD normal basta. Tampoco se separan bases de datos, porque el volumen del negocio no lo justifica (ADR-004).

Etapa 7. Revisar la propuesta con SOLID

Se revisó la propuesta completa antes de cerrar los diagramas. Se registraron siete hallazgos: cinco obligaron a cambiar algo y dos confirman decisiones.

#	Principio	Evidencia encontrada	Impacto	Cambio realizado
S-1	S	Agrupar por Aggregate juntaba en un ServicioAppService lo que hacen tres roles.	Un cambio en facturación tocaba la clase del supervisor (contra RB-15).	Se separó por actor: Programacion, EjecucionServicio y Facturacion AppService.
S-2	O	CalculadoraDePrecio con if por tipo de inmueble y el 10% fijo en el método.	Cambiar una tarifa o agregar un tipo obligaba a editar el cálculo.	Tarifas y descuento se inyectan (TarifasPorTipoDeInmueble, PoliticaDeDescuento).
S-3	O / S	RegistrarIncidencia con un switch donde «acceso denegado» cambiaba el estado.	Cada tipo con efecto especial alargaba el switch.	Acceso denegado se volvió su propio método y Command.
S-4	L	Las implementaciones en memoria y EF deben comportarse igual.	Las pruebas podrían pasar y producción fallar.	Confirmación con contrato documentado: null si no existe y nada nuevo se guarda hasta confirmar.
S-5	I	IServicioRepository tenía métodos de reportes.	Los servicios de escritura dependían de métodos que no usan.	Los reportes pasaron a IConsultasDeServicios e IConsultasDeFacturacion.
S-6	S / D	La validación SUP-04 estaba en FacturacionAppService.	Regla del negocio en la capa que solo coordina.	Se movió a GeneradorDeFactura (dominio) usando IFacturaRepository.
S-7	D	Dominio y aplicación solo conocen interfaces; la notificación no depende de correo.	Se puede cambiar la base de datos o el medio de aviso sin tocar el dominio.	Confirmación: CleanPro.Domain no referencia a ningún proyecto.

Tabla 5. *Hallazgos de la revisión SOLID.*

Etapa 8. Representar y documentar la arquitectura

Los diagramas se elaboraron en draw.io; los archivos fuente (.drawio) y sus exportaciones (.png) están en la carpeta diagrams del repositorio. El diagrama de clases (Figura 2) separa visualmente las capas de aplicación, dominio e infraestructura. Los diagramas de secuencia muestran el flujo principal de CU-03 (Figura 3) y el flujo de facturación con sus reglas, el rechazo y la alternativa por servicio no realizado (Figura 4). Los participantes de las secuencias son las mismas clases del diagrama de clases, con los mismos nombres de métodos.

Architecture Decision Records (ADR)

Se elaboraron cuatro ADR (la guía pide al menos tres). Cada uno contiene título, estado, contexto, alternativas, decisión, consecuencias y trazabilidad; están completos en docs/adr del repositorio.

ADR	Decisión	Alternativas consideradas	Consecuencia principal	Trazabilidad
ADR-001	Domain Model para las operaciones de escritura	Transaction Script, Table Module	Reglas en un solo lugar; más clases y necesidad de Repository/UoW	RB-02, RB-03, RB-07, RB-14, PA-02
ADR-002	Repository por Aggregate + Unit of Work; un Aggregate por transacción; eventos al confirmar	Acceso directo a datos; repositorio genérico CRUD	Incidencia y aviso se guardan juntos (PA-01); riesgo de concurrencia en cuadrillas	RB-06, RES-01, RES-04, BN-06
ADR-003	Incidencia como Entity dentro del Aggregate Servicio	Incidencia como Aggregate propio	RB-14 y RES-03 se validan en un solo lugar; el servicio se carga completo	RB-11, RB-12, RB-14, RES-03, SUP-03
ADR-004	CQRS básico: Commands por el dominio, Queries con DTOs	Leer con los Aggregates; CQRS con bases separadas	Repositorios pequeños; más clases de lectura	REP-02, REP-04, REP-05, US-17, US-21

Tabla 6. *Resumen de los registros de decisión de arquitectura.*

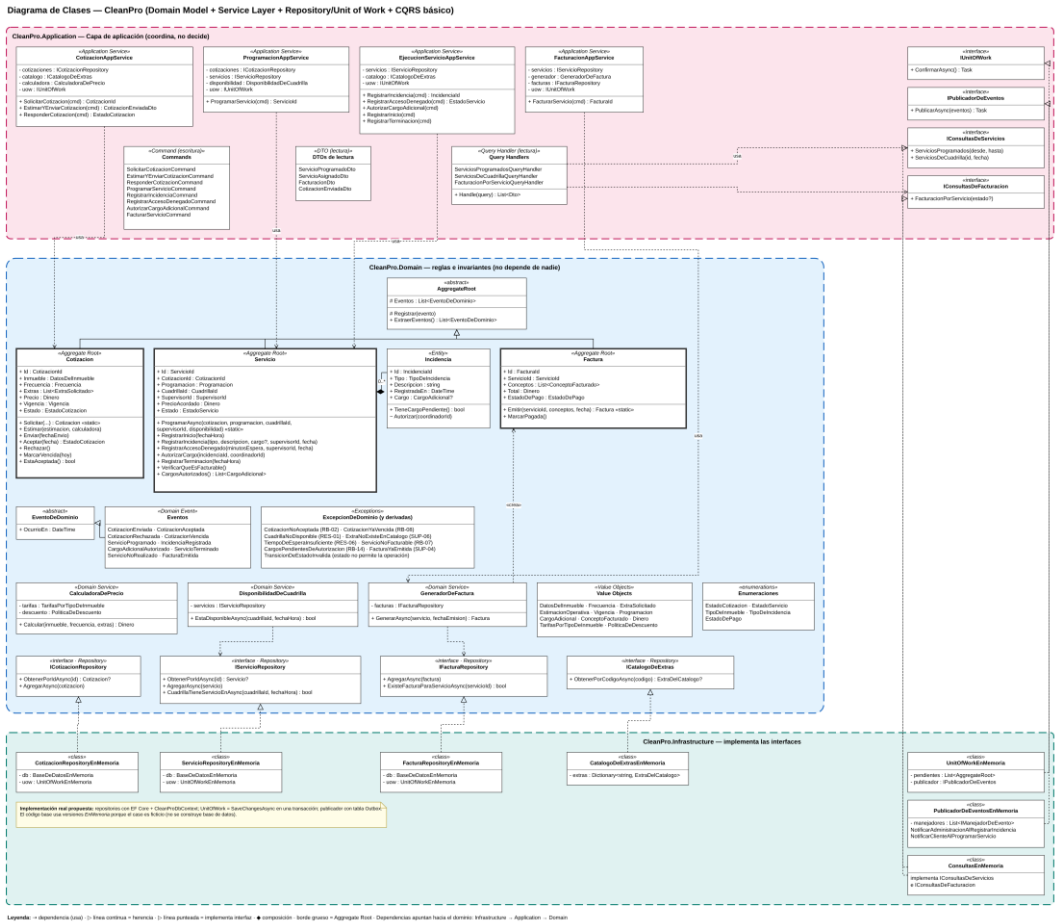


Figura 2. *Diagrama de clases por capas (diagrams/src/02-diagrama-clases.drawio).*

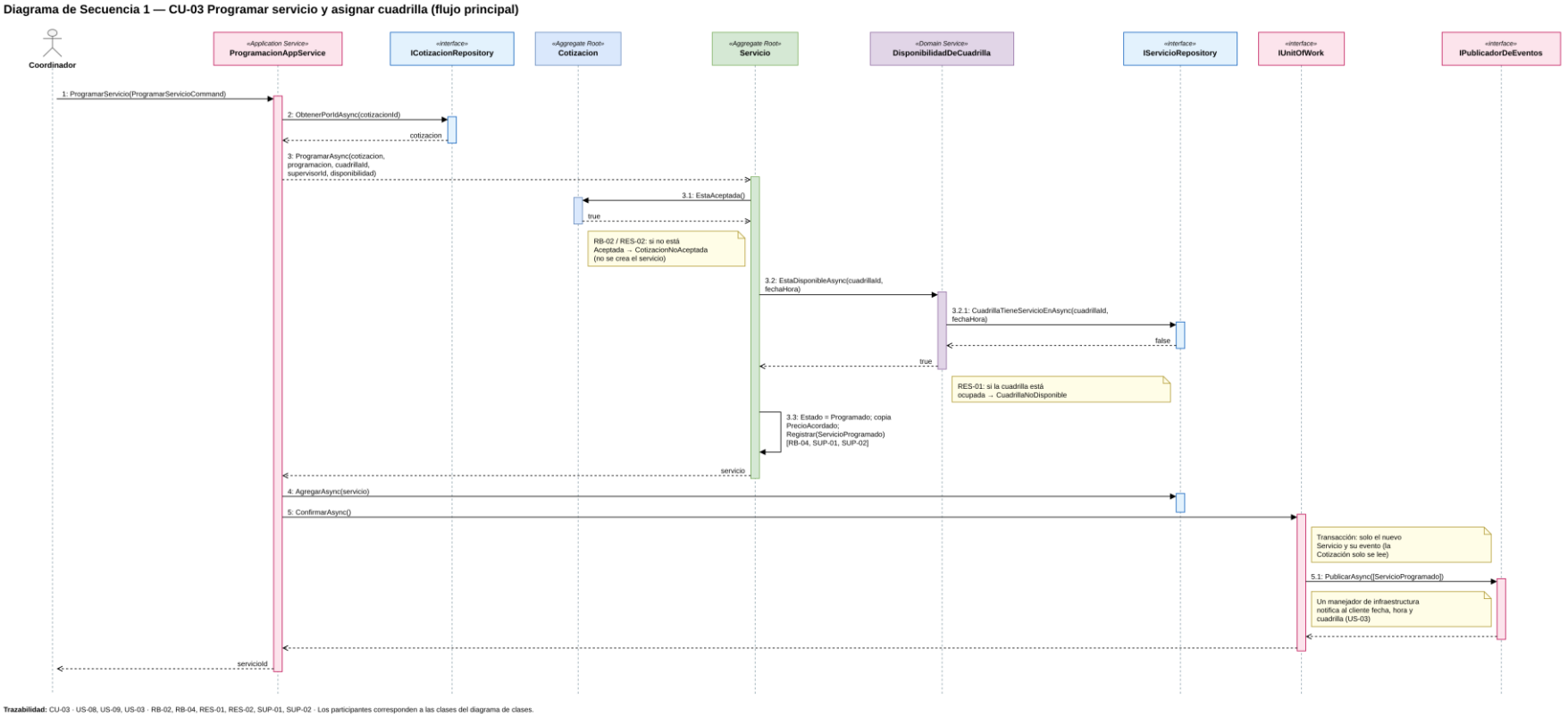


Figura 3. *Diagrama de secuencia 1 — CU-03 Programar servicio y asignar cuadrilla (flujo principal).*

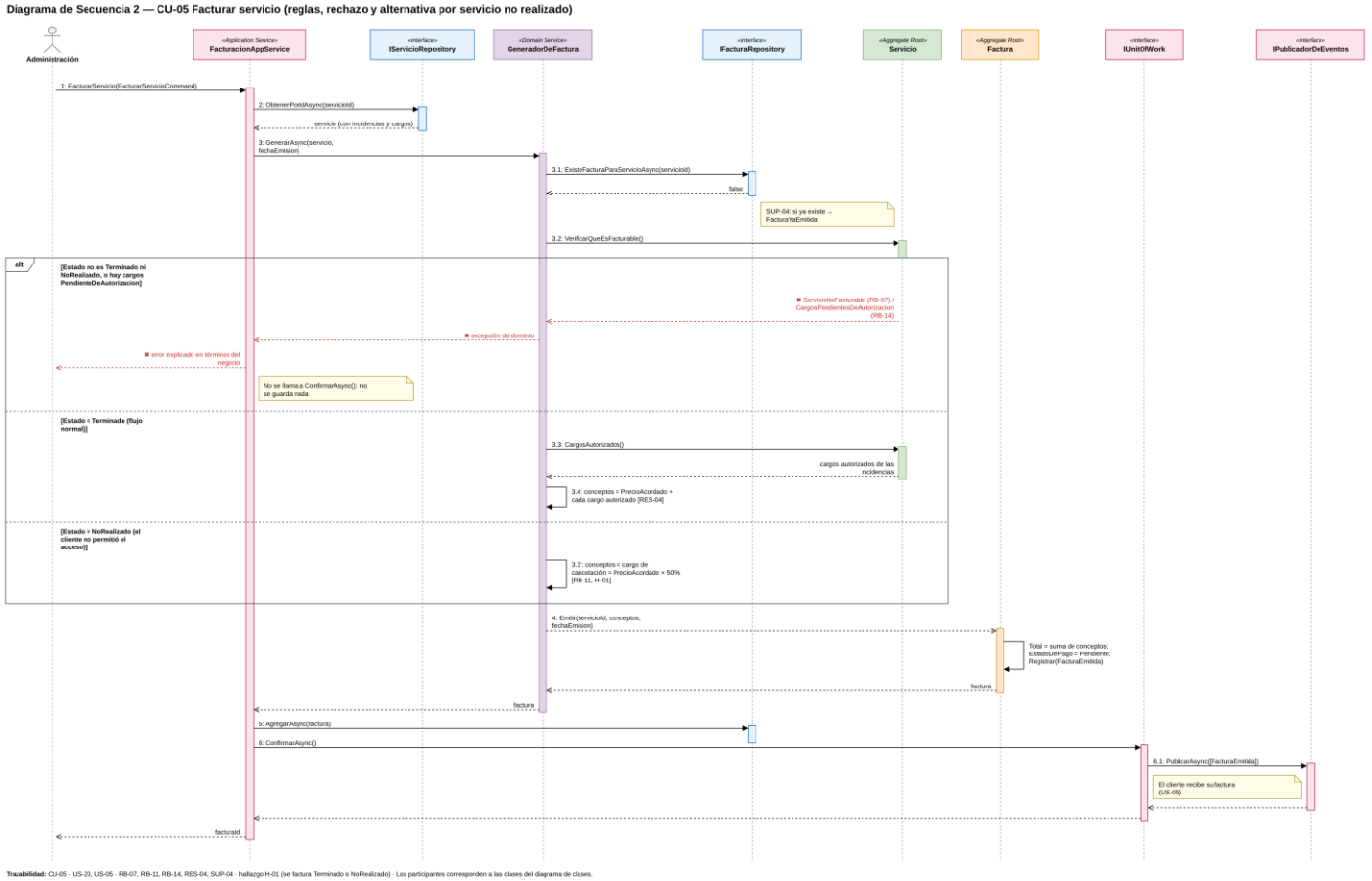


Figura 4. *Diagrama de secuencia 2 — CU-05 Facturar servicio (reglas, rechazo y servicio no realizado).*

Código base

Se generó una solución en C# (.NET 8) con cuatro proyectos que siguen las capas del diagrama de clases: CleanPro.Domain (sin dependencias), CleanPro.Application, CleanPro.Infrastructure y CleanPro.Tests. Las dependencias apuntan hacia el dominio: Infrastructure → Application → Domain. La Figura 5 muestra la estructura de carpetas y la Figura 6 el Domain Service que aplica las reglas de facturación.

Estructura de la solución CleanPro (C# / .NET 8)

```
cleanpro-arquitectura-u3/
├── CleanPro.sln
├── README.md
├── docs/  (u2, adr, matriz, bitácora, reporte)
├── diagrams/  (src .drawio, png)
├── src/
│   ├── CleanPro.Application/
│   │   ├── Comun/
│   │   │   └── Puertos.cs
│   │   ├── Consultas/
│   │   │   ├── Consultas.cs
│   │   │   └── QueryHandlers.cs
│   │   ├── Cotizaciones/
│   │   │   ├── Commands.cs
│   │   │   └── CotizacionAppService.cs
│   │   ├── Ejecucion/
│   │   │   ├── Commands.cs
│   │   │   └── EjecucionServicioAppService.cs
│   │   ├── Facturacion/
│   │   │   └── FacturacionAppService.cs
│   │   ├── ProgramacionDeServicios/
│   │   │   └── ProgramacionAppService.cs
│   │   └── CleanPro.Application.csproj
│   ├── CleanPro.Domain/
│   │   ├── Comun/
│   │   │   ├── AggregateRoot.cs
│   │   │   ├── Dinero.cs
│   │   │   ├── Excepciones.cs
│   │   │   └── Identificadores.cs
│   │   ├── Cotizaciones/
│   │   │   ├── CalculadoraDePrecio.cs
│   │   │   ├── CatalogoDeExtras.cs
│   │   │   ├── Cotizacion.cs
│   │   │   ├── Eventos.cs
│   │   │   ├── ICotizacionRepository.cs
│   │   │   └── ValueObjects.cs
│   │   ├── Facturacion/
│   │   │   ├── Eventos.cs
│   │   │   ├── Factura.cs
│   │   │   ├── GeneradorDeFactura.cs
│   │   │   └── IFacturaRepository.cs
│   │   ├── Servicios/
│   │   │   ├── DisponibilidadDeCuadrilla.cs
│   │   │   ├── Eventos.cs
│   │   │   ├── IServicioRepository.cs
│   │   │   ├── Incidencia.cs
│   │   │   ├── Servicio.cs
│   │   │   └── ValueObjects.cs
│   │   └── CleanPro.Domain.csproj
│   └── CleanPro.Infrastructure/
│       ├── EnMemoria/
│       │   ├── BaseDeDatosEnMemoria.cs
│       │   ├── CatalogoDeExtrasEnMemoria.cs
│       │   ├── ConsultasEnMemoria.cs
│       │   ├── PublicadorDeEventosEnMemoria.cs
│       │   ├── Repositorios.cs
│       │   └── UnitOfWorkEnMemoria.cs
│       ├── CleanPro.Infrastructure.csproj
│       └── SistemaCleanProEnMemoria.cs
└── tests/
    └── CleanPro.Tests/
        ├── CleanPro.Tests.csproj
        ├── CotizacionTests.cs
        ├── EjecucionTests.cs
        ├── Escenario.cs
        ├── FacturacionTests.cs
        └── ProgramacionTests.cs
```

Figura 5. *Estructura inicial de carpetas y código de la solución.*

src/CleanPro.Domain/Facturacion/GeneradorDeFactura.cs

```
public async Task<Factura> GenerarAsync(Servicio servicio, DateTime fechaEmision, CancellationToken ct = default)
{
    if (await _facturas.ExisteFacturaParaServicioAsync(servicio.Id, ct))
        throw new FacturaYaEmitida();

    servicio.VerificarQueEsFacturable();

    var conceptos = new List<ConceptoFacturado>();
    if (servicio.Estado == EstadoServicio.Terminado)
    {
        conceptos.Add(new ConceptoFacturado("Servicio de limpieza", servicio.PrecioAcordado, OrigenDelConcepto.Servicio));
        conceptos.AddRange(servicio.CargosAutorizados().Select(cargo =>
            new ConceptoFacturado("Cargo adicional autorizado", cargo.Monto, OrigenDelConcepto.Incidencia)));
    }
    else
    {
        conceptos.Add(new ConceptoFacturado(
            $"Cargo de cancelación por falta de acceso ({PorcentajeCargoCancelacion}%)",
            servicio.PrecioAcordado.Porcentaje(PorcentajeCargoCancelacion),
            OrigenDelConcepto.Cancelacion));
    }

    return Factura.Emitir(servicio.Id, conceptos, fechaEmision);
}
```

Figura 6. *GeneradorDeFactura: aplica SUP-04, RB-07, RB-14, RES-04 y el cargo de cancelación de RB-11.*

Las pruebas automatizadas (xUnit) recorren las reglas de negocio de los cinco casos de uso; cada prueba lleva en su nombre la regla que verifica:

Clase de prueba	Escenarios
CotizacionTests (6)	Precio residencial único = $1,200 (RB-03); empresarial semanal con extra = $1,550 (RB-03, H-05); extra inexistente (SUP-06); aceptar dentro de vigencia; aceptar fuera de vigencia → Vencida (RB-08); rechazar.
ProgramacionTests (3)	Flujo principal con aviso al cliente (US-03); no programar sin aceptación (RB-02); cuadrilla ocupada (RES-01).
EjecucionTests (4)	Incidencia notificada a administración (RB-06, PA-01); servicio adicional con cargo pendiente (RB-12); espera insuficiente (RES-06); acceso denegado → No realizado (RB-11).
FacturacionTests (7)	Terminado = precio + cargos autorizados (RES-04); no facturar con cargo pendiente (RB-14); no facturar en ejecución (RB-07); no realizado = 50% (RB-11, H-01); una factura por servicio (SUP-04); consultas de facturación y de servicios de la cuadrilla (CQRS).

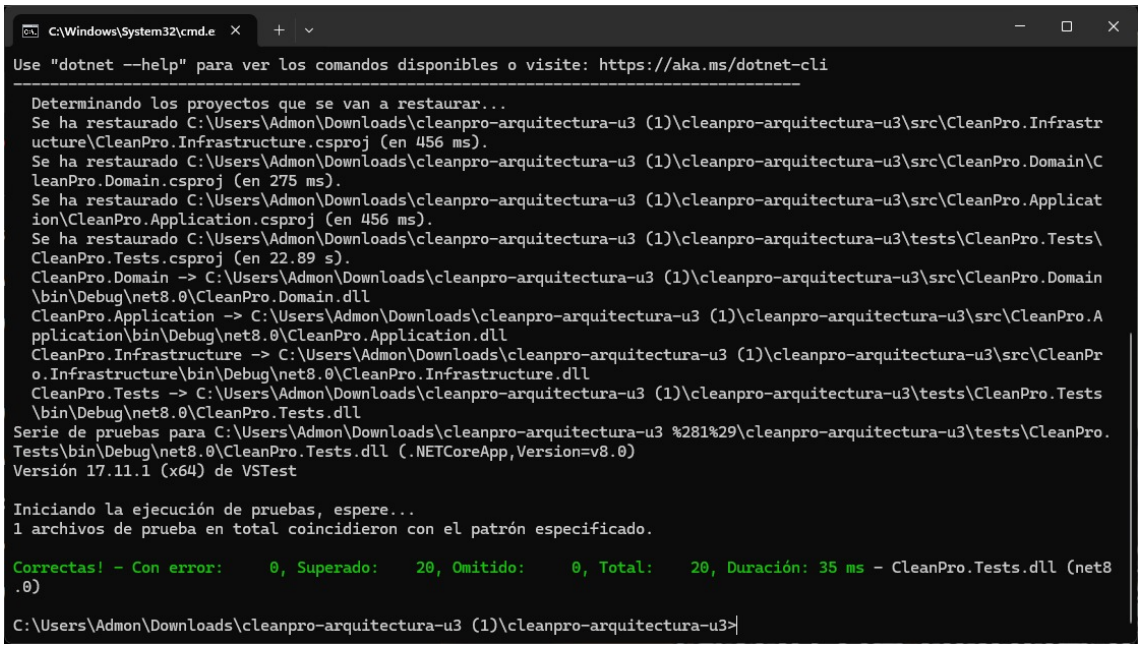


Figura 7. *Resultado de «dotnet test»: 20 pruebas correctas, 0 con error.*

Etapa 9. Validar la propuesta

Se recorrió cada caso de uso sobre los diagramas y el código para identificar qué componente recibe la solicitud, dónde se valida cada regla, qué objetos cambian, cómo se persisten y qué se devuelve.

Caso de uso	Recibe	Dónde se valida cada regla	Objetos que cambian	Se devuelve
CU-01	CotizacionAppService	SUP-06 → ICatalogoDeExtras; RB-03 → CalculadoraDePrecio; RB-01 → Cotizacion.Enviar	Cotización (Solicitada → Pendiente)	CotizacionId / CotizacionEnviadaDto
CU-02	CotizacionAppService	RB-08 → Vigencia; estado Pendiente → Cotizacion	Cotización (Aceptada / Rechazada / Vencida)	EstadoCotizacion
CU-03	ProgramacionAppService	RB-02, RB-04 → Servicio.ProgramarAsync; RES-01 → DisponibilidadDeCuadrilla	Nuevo Servicio	ServicioId
CU-04	EjecucionServicioAppService	RB-06, RB-12, RB-14 → Servicio.RegistrarIncidencia; RB-11, RES-06 → RegistrarAccesoDenegado	Servicio, Incidencia, CargoAdicional	IncidencialId / EstadoServicio
CU-05	FacturacionAppService	SUP-04, RB-11 → GeneradorDeFactura; RB-07, RB-14 → Servicio; RES-04 → Factura	Nueva Factura	FacturaId

Validación	Pregunta	Resultado del equipo
Trazabilidad	¿Cada componente importante responde a una necesidad, regla o caso de uso?	Sí. La matriz liga BN-01 a BN-10 con reglas, casos de uso, clases, ADR y pruebas; las historias no diseñadas están listadas con su motivo.
Lenguaje	¿Los mismos términos significan lo mismo en BRD, UML, código y ADR?	Sí. Los nombres salen del glosario; se agregaron siete términos y se unificaron los nombres de métodos entre diagramas y código.
Responsabilidad	¿Las reglas permanecen en el dominio?	Sí. Los Application Services no tienen reglas; SUP-04 se movió al dominio (S-6).
Consistencia	¿Los diagramas de clase y secuencia representan la misma arquitectura?	Sí. Los participantes de ambas secuencias existen en el diagrama de clases con los mismos métodos.
Persistencia	¿El dominio evita depender de SQL, ORM o infraestructura?	Sí. CleanPro.Domain no referencia a ningún proyecto; los repositorios son interfaces.
Transacción	¿El límite de la operación y su manejo de error están definidos?	Sí. Cada caso de uso termina en ConfirmarAsync y modifica un solo Aggregate; si el dominio rechaza no se guarda nada (escenarios NoFacturarConCargoPendiente y NoProgramarSinAceptacion).
IA	¿Cada aporte de IA fue revisado y su uso está registrado?	Sí. La bitácora registra nueve iteraciones con sus supuestos, errores detectados, correcciones y la decisión del equipo.

¿El diseño responde a las necesidades? Sí: cada uno de los cuatro problemas del BRD tiene un mecanismo concreto y al menos una prueba que lo demuestra (ver la matriz de trazabilidad). **Ajustes pendientes:** validar con el negocio los supuestos H-04, H-05 y H-06 y la definición de «cuadrilla disponible»; diseñar la reprogramación (RB-09) y la reasignación de personal (RB-10); y agregar control de concurrencia en la implementación real.

Matriz de trazabilidad actualizada

Necesidad	User Stories	Reglas	CU	Componentes	Decisiones	Pruebas
BN-01 Gestionar la solicitud y el cálculo de cotizaciones	US-01, US-06, US-07, US-22	RB-01, RB-03, RB-08, RB-15, SUP-06	CU-01	Cotizacion, CalculadoraDePrecio, TarifasPorTipoDeInmueble, PoliticaDeDescuento, ICatalogoDeExtras, CotizacionAppService	ADR-001, SOLID S-2, H-04, H-05	PrecioResidencialUnico, PrecioEmpresarialRecurrenteConExtra, ExtraInexistente
BN-02 Registrar la aceptación o rechazo de la cotización	US-02, US-22	RB-02, RB-08, RES-05	CU-02	Cotizacion.Aceptar / Rechazar / MarcarVencida, Vigencia	ADR-001, H-03	AceptarDentroDeVigencia, AceptarFueraDeVigencia, Rechazar
BN-03 Programar los servicios aceptados y asignar cuadrilla	US-03, US-08, US-09	RB-02, RB-04, RES-01, RES-02, SUP-01, SUP-02	CU-03	Servicio.ProgramarAsync, DisponibilidadDeCuadrilla, ProgramacionAppService, NotificarClienteAlProgramarServicio	ADR-002, Secuencia 1	ProgramarFlujoPrincipal, NoProgramarSinAceptacion, CuadrillaOcupada
BN-04 Registrar el inicio y la terminación de cada servicio	US-13, US-14, US-17	RB-05, RES-03	Soporte de CU-04 y CU-05	Servicio.RegistrarInicio / RegistrarTerminacion, EjecucionServicioAppService, ServiciosDeCuadrillaQueryHandler	ADR-004	ConsultaDeServiciosDeCuadrilla

Necesidad	User Stories	Reglas	CU	Componentes	Decisiones	Pruebas
BN-05 Registrar incidencias y cambios durante el servicio	US-11, US-15, US-16	RB-06, RB-11, RB-12, RB-14, RES-06, SUP-03	CU-04	Servicio.RegistrarIncidencia / RegistrarAccesoDenegado / AutorizarCargo, Incidencia, CargoAdicional, EjecucionServicioAppService	ADR-003, SOLID S-3, H-02	ServicioAdicionalQuedaPendiente, EsperaInsuficiente, AccesoDenegado
BN-06 Notificar oportunamente a administración	US-19	RB-06	CU-04	IncidenciaRegistrada, IUnitOfWork, IPublicadorDeEventos, NotificarAdministracionAlRegistrarIncidencia	ADR-002	IncidenciaNotificaAAdministracion
BN-07 Registrar y dar seguimiento a la facturación	US-05, US-20	RB-07, RB-11, RB-14, RES-04, SUP-04	CU-05	Factura, GeneradorDeFactura, IFacturaRepository, FacturacionAppService	ADR-001, SOLID S-6, H-01, Secuencia 2	FacturaTerminadoConCargoAutorizado, NoFacturarConCargoPendiente, NoFacturarEnEjecucion, FacturaNoRealizado, UnaFacturaPorServicio
BN-08 Mantener evidencia trazable de incidencias y extras	US-11, US-15	RB-06, RB-12, RB-14	CU-04, CU-05	Incidencia dentro de Servicio, ConceptoFacturado.Origen, Domain Events	ADR-003	FacturaTerminadoConCargoAutorizado

Necesidad	User Stories	Reglas	CU	Componentes	Decisiones	Pruebas
BN-09 Reducir la asignación manual de cuadrillas	US-09, US-10	RES-01 (RB-10 posterior)	CU-03	DisponibilidadDeCuadrilla, IServicioRepository.CuadrillaTieneServicioEnAsync	ADR-002 (riesgo de concurrencia)	CuadrillaOcupada
BN-10 Generar reportes para administración	US-21	REP-02, REP-04, REP-05	Queries (CQRS)	ServiciosProgramadosQueryHandler, ServiciosDeCuadrillaQueryHandler, FacturacionPorServicioQueryHandler, IConsultasDeServicios, IConsultasDeFacturacion, ConsultasEnMemoria	ADR-004, SOLID S-5	ConsultaDeFacturacion, ConsultaDeServiciosDeCuadrilla

Tabla 7. *Matriz de trazabilidad: necesidad → historia → regla → caso de uso → componente → decisión → prueba.*

Bitácora de uso de inteligencia artificial

Herramienta utilizada: Claude (Anthropic). Cada iteración registra el objetivo, el prompt, lo útil de la respuesta, los errores o supuestos detectados y la decisión que tomó el equipo. También se entrega por separado en docs/bitacora-ia.pdf.

#	Objetivo	Prompt (resumen)	Respuesta útil	Error o supuesto detectado	Decisión del equipo
1	Entender la actividad y los entregables	«¿Qué documentos necesitas y qué tengo que subir?» + guía del docente + transcripción del video	Lista de entregables, orden de trabajo y estructura del repositorio	La IA propuso PlantUML (.puml) para los diagramas. También fue necesario aclarar que el informe PDF de la guía y el Reporte FIME son el mismo documento.	Se decidió usar draw.io (.drawio) y entregar un solo Reporte FIME que cubre lo que piden la guía y el docente.
2	Seleccionar casos de uso	«Con el BRD, User Stories, Glosario y BPMN selecciona 3-5 casos de uso con al menos un flujo de rechazo»	5 CU con flujos normal, alterno y de rechazo; 6 hallazgos de continuidad	Detectó la contradicción RB-07 vs RB-11 (H-01) y estados faltantes (H-02, H-03). H-04, H-05 y H-06 eran supuestos de la IA, no reglas del BRD.	Se aceptaron los 5 CU; US-04 y US-10 quedan como trabajo posterior. Se aceptaron H-04 (en la entrevista no se dio fórmula de duración y personal) y H-05 como supuestos.

#	Objetivo	Prompt (resumen)	Respuesta útil	Error o supuesto detectado	Decisión del equipo
3	Elegir el patrón	«Compara Transaction Script, Table Module y Domain Model con las reglas RB-01..RB-15. Declara supuestos»	Tabla comparativa con evidencias; Domain Model + consultas simples; ADR-001	Se revisó que la comparación no agregara reglas que no están en el BRD.	Se aceptó Domain Model para las operaciones que cambian el estado y consultas simples para los reportes (ADR-001).
4	Mapa de dominio	«Propón entidades, VO, Aggregates, Domain Services y Events con evidencia y dudas»	3 Aggregates, 9 VO, 3 Domain Services, 10 eventos; diagrama draw.io	La IA agregó el tipo «OtroCambio» para cubrir «cualquier incidencia o cambio» de RB-06. Decisiones dudosas: Incidencia dentro de Servicio; Cuadrilla como referencia.	Se aprobó el mapa. Se mantuvo «OtroCambio» porque RB-06 sí habla de cambios, e Incidencia dentro de Servicio (ADR-003).
5	Service Layer, Repository/UoW y CQRS	«Propón firmas que solo coordinen, repositorios en términos del dominio y separa Commands/Queries»	4 servicios por actor, 8 Commands, 3 Queries, 4 repositorios	Para no pasar de 5 CU, CU-01 quedó con 2 métodos y CU-04 con 3; autorizar un cargo no era CU propio, pero CU-05 depende de él.	Se aceptó agrupar por actor y que CU-01 y CU-04 tengan más de un método, porque cada uno es una intención distinta del usuario.

#	Objetivo	Prompt (resumen)	Respuesta útil	Error o supuesto detectado	Decisión del equipo
6	Revisión SOLID	«Señala posibles violaciones SOLID con evidencia, riesgo y alternativas; no reescribas todo»	7 hallazgos (S-1 a S-7)	La validación SUP-04 había quedado en el Application Service (versión del paso 3).	Se aceptaron los 7 hallazgos; SUP-04 se movió al dominio y se actualizó el mapa de dominio.
7	Diagramas UML	«Convierte la arquitectura validada en diagramas draw.io; no agregues clases ni reglas»	Diagrama de clases y dos diagramas de secuencia	Se verificó que los participantes de las secuencias coincidieran con las clases.	Se aprobaron el contenido y la elección de CU-03 (flujo principal) y CU-05 (rechazo y alternativa) para las secuencias.
8	Corregir diagramas	«Que los textos no se encimen sobre líneas, barras y notas»	Diagramas reacomodados sin encimados	El equipo detectó textos encimados en la primera versión. En la revisión aparecieron nombres de métodos distintos entre diagramas (Programar vs ProgramarAsync).	Se aprobaron las versiones corregidas y se unificaron los nombres en todos los diagramas.

#	Objetivo	Prompt (resumen)	Respuesta útil	Error o supuesto detectado	Decisión del equipo
9	Código base C#	«Genera la estructura y código mínimo en C# que respete los diagramas validados»	4 proyectos, 41 archivos, 20 pruebas xUnit	La IA no pudo compilar en su entorno, así que las pruebas se ejecutaron con «dotnet test» en la computadora del equipo. Se renombró una excepción que chocaba con un evento del mismo nombre. Los extras del catálogo son datos de ejemplo.	Se eligió C# y repositorios en memoria porque el caso es ficticio.

Conclusiones:

Esta actividad permitió comprobar para qué servía el levantamiento de la Unidad 2: cada decisión de arquitectura se pudo justificar con una necesidad, una regla o una historia concreta del BRD de CleanPro. El patrón no se eligió por ser el más moderno, sino porque el negocio tiene reglas que dependen del estado de cada cotización y de cada servicio, y porque su problema principal —los errores de facturación— nace de información dispersa. Con Domain Model esas reglas quedaron en un solo lugar: la factura solo se arma desde un servicio que ya validó sus incidencias y sus cargos.

El trabajo también obligó a revisar el propio análisis. Al cruzar los documentos se encontró una contradicción entre RB-07 y RB-11 y estados que faltaban en el BRD; registrarlos como hallazgos, en lugar de inventar reglas, mantuvo la coherencia entre el BRD, los diagramas y el código. La revisión con SOLID tuvo efectos concretos: separó los servicios por actor, sacó los reportes de los repositorios y regresó al dominio una regla que se había quedado en la capa de aplicación. Las pruebas automatizadas muestran que el diseño protege las reglas del negocio.

La inteligencia artificial fue útil para proponer alternativas, detectar inconsistencias y generar borradores de diagramas y código, pero cada propuesta tuvo que contrastarse con el BRD. Algunos supuestos de la IA tuvieron que validarse, la primera versión de los diagramas se corrigió y el código se verificó ejecutando las pruebas. El resultado es una arquitectura que el equipo puede explicar y defender, con ajustes pendientes identificados: reprogramación, reasignación de personal y control de concurrencia en la asignación de cuadrillas.

Conclusiones individuales:

Roberto Carlos Sánchez Flores

En esta actividad entendí para qué servía todo el levantamiento que hicimos en la Unidad 2. Al principio pensaba que elegir el patrón era cuestión de escoger el más completo, pero al comparar Transaction Script, Table Module y Domain Model con las reglas de CleanPro me di cuenta de que la decisión tenía que salir del negocio: las reglas de facturación dependen del estado de cada servicio y de sus incidencias, y eso es justo lo que Domain Model protege. También aprendí a leer con más cuidado nuestros propios documentos; encontrar la contradicción entre RB-07 y RB-11 me enseñó que una arquitectura no se puede construir sobre reglas que se contradicen, y que lo correcto es registrar el hallazgo y corregirlo en lugar de acomodar el diseño a la fuerza.

Donovan Adrian Serrato Lores

Lo que más me dejó esta actividad fue separar responsabilidades. Antes habría puesto casi todo en una sola clase de servicio; ahora entiendo que la Service Layer solo debe coordinar y que las reglas tienen que vivir en el dominio. La revisión con SOLID fue donde más lo noté: al revisar el diseño vimos que un solo servicio mezclaba lo que hacen el coordinador, el supervisor y administración, que los reportes no debían estar en los repositorios y que la regla de una factura por servicio estaba en la capa equivocada. También entendí para qué sirven la Unit of Work y los Domain Events: gracias a ellos una incidencia y su aviso a administración se guardan juntos, que era uno de los problemas principales de la empresa.

Laura Sofía Vargas Ramírez

Para mí lo más importante fue mantener el mismo lenguaje en todo el trabajo. Los términos del glosario se convirtieron en clases, métodos y diagramas, y la matriz de trazabilidad me ayudó a comprobar que cada componente responde a una necesidad o a una regla del BRD. También aprendí a usar la inteligencia artificial como apoyo y no como respuesta final: algunas propuestas traían supuestos que no estaban en nuestros documentos, los primeros diagramas tenían textos encimados y hubo nombres de métodos que no coincidían entre un diagrama y otro. Revisar cada resultado contra el BRD y dejarlo registrado en la bitácora me hizo ver que la responsabilidad del diseño sigue siendo del equipo.

Repositorio de GitHub:

Todos los entregables están en el repositorio del equipo: docs (informe, ADR, matriz de trazabilidad, bitácora de IA e insumos de la Unidad 2), diagrams (fuentes .drawio y exportaciones), src (dominio, aplicación e infraestructura), tests y un README con instrucciones de revisión.

Enlace del GitHub:

https://github.com/RobertocSanchezf23/CleanPro_Equipo12